Architecting the Virtual Machine: A Comprehensive Guide to Stack-Based Bytecode Interpreters

1. Foundations of Process Virtualization

The construction of a virtual machine (VM) is a fundamental rite of passage in systems programming. It serves as the bridge between the abstract logic of high-level software and the concrete, rigid reality of hardware execution. Your upcoming assignment, detailed in the "Lab 4" specification , tasks you with building a stack-based bytecode virtual machine. This is not merely an exercise in C programming; it is an architectural challenge that requires a deep, nuanced understanding of how computers process information at the lowest level.

To understand the virtual machine, one must first understand the problem it solves. In the early days of computing, software was inextricably linked to the hardware it ran on. A program written for a specific mainframe could not run on a minicomputer from a different vendor without a complete rewrite. The compiler translated source code directly into the native machine code of the target processor (x86, ARM, MIPS, etc.). This approach, while performant, created a fragmentation problem.

The Process Virtual Machine—the type you are tasked to build—introduces an intermediate layer. Instead of compiling to native machine code, the compiler targets an idealized, abstract machine. This abstract machine has its own instruction set, its own memory model, and its own rules of execution. The code produced for this machine is called **bytecode**. The VM is simply a program running on the host hardware that simulates this abstract machine, interpreting the bytecode instructions one by one and performing the corresponding actions on the physical CPU.

This "Write Once, Run Anywhere" philosophy, popularized by UCSD Pascal and later solidified by Java and the JVM, relies entirely on the efficiency and robustness of the VM implementation. In this report, we will dissect the anatomy of such a machine. We will explore the theoretical underpinnings of stack architectures, the mechanical implementation of the fetch-decode-execute cycle, the intricate dance of memory management during function calls, and the tooling required to bring the machine to life.

1.1 The Systems Perspective

From a systems engineering perspective, a VM is a software CPU. Where a physical CPU is constructed from logic gates and transistors etched into silicon, your VM will be constructed from data structures and control flow statements written in C.

- **Registers** become variables (e.g., uint32_t pc).
- **Memory** becomes arrays (e.g., int32_t stack).
- **The Clock** becomes a while loop.
- **The Control Unit** becomes a switch statement.

Understanding this mapping is crucial. Every design choice you make in software has a parallel in hardware design, and understanding the hardware constraints (cache locality, branch

prediction, memory alignment) will make you a better VM architect.

2. Architectural Paradigms: Stack vs. Register Machines

Before writing the first line of code, an architect must decide on the fundamental model of computation. The two dominant paradigms for VM design are the **Stack-Based Architecture** and the **Register-Based Architecture**. Your assignment mandates a stack-based design , but to truly master it, you must understand why this choice was made and what trade-offs it entails compared to its alternative.

2.1 The Stack-Based Model

In a stack machine, there are no general-purpose registers for storing temporary values. All computation takes place on a central Data Stack. The operands for any operation are implicitly located at the top of this stack.

Mechanism of Operation: Consider the arithmetic operation A = B + C. In a stack machine, this logic flow proceeds as follows:

1. **PUSH B**: The value of B is loaded from memory and placed on top of the stack.
2. **PUSH C**: The value of C is placed on top of the stack (pushing B down).
3. **ADD**: The instruction removes (pops) the top two elements (C and B), adds them together in the ALU (Arithmetic Logic Unit), and pushes the result (A) back onto the stack.
4. **POP A**: The value at the top of the stack (the result) is removed and stored in the memory location for variable A.

Advantages:

- **Zero-Address Instructions:** Instructions like ADD, MUL, or SUB do not need to encode any operands. They automatically apply to the top elements. This makes the bytecode extremely compact (often just 1 byte per opcode).
- **Compiler Simplicity:** Generating code for a stack machine is trivial. It directly corresponds to a post-order traversal of the Abstract Syntax Tree (AST). For the expression (a + b) * c, the traversal yields PUSH a, PUSH b, ADD, PUSH c, MUL. No complex register allocation algorithms are needed.
- **Hardware Independence:** The model assumes an infinite stack depth, freeing the compiler writer from worrying about the specific number of registers available on the target architecture.

Disadvantages:

- **Instruction Overhead:** Stack machines generally require more instructions to perform a task than register machines. The constant shuffling of data to and from the stack (Push/Pop cycles) generates a higher instruction count.
- **Memory Traffic:** In a software implementation, the stack is an array in RAM. Every operation requires reading and writing to this array. This can create a bottleneck if the host CPU's cache is not utilized effectively, although modern caches handle linear array access quite well.

2.2 The Register-Based Model A register-based VM (like the Lua VM or Android's Dalvik) simulates a CPU with a specific set of virtual registers (e.g., v0 through v15).

Mechanism of Operation: For A = B + C:

1. **ADD vA, vB, vC**: The instruction explicitly commands the machine to add the contents of

register B and C and store the result in register A.

Advantages:

- **Efficiency:** The operation is completed in a single instruction, reducing the overhead of the fetch-decode loop. There is no temporary data movement; the calculation happens "in place".
- **Performance:** Register VMs often map more efficiently to the host machine's physical registers, especially when using Just-In-Time (JIT) compilation.

Disadvantages:

- **Complexity:** The instructions are larger because they must carry the IDs of the registers (e.g., 3 operands).
- **Compiler Difficulty:** The compiler must perform "Register Allocation" (often using graph coloring algorithms) to map the potentially infinite variables of a user's program to the finite set of virtual registers.

2.3 Why Stack-Based for Systems Education?

Your assignment selects the stack-based model because it isolates the *systems* concepts from the *compiler* concepts. By removing the need for register allocation, you can focus entirely on the memory layout, the fetch-execute cycle, and the binary representation of instructions. It is the purest way to learn how a machine executes code. The complexity of the implementation is shifted from the compiler (which is hard) to the interpreter (which is straightforward).

3. The Instruction Cycle: The Heartbeat of the Machine

At the core of every processor is the **Fetch-Decode-Execute (FDE)** cycle. This is an infinite loop that drives the machine forward. Understanding this cycle is non-negotiable.

3.1 The Registers

Even a stack machine needs a few internal registers to manage its state. In C, these will be variables within your VM struct.

1. **Program Counter (PC) / Instruction Pointer (IP):** This holds the memory address (or array index) of the *next* instruction to be executed. It is the machine's cursor, pointing to its current place in the code.
2. **Stack Pointer (SP):** This holds the index of the top element of the data stack. It moves up and down as data is pushed and popped.
3. **Frame Pointer (FP):** This holds the index of the base of the current stack frame. It is essential for accessing local variables and function arguments relative to the current function scope.
4. **Instruction Register (IR):** In hardware, this holds the current instruction bit pattern. In your software VM, this is simply a local variable holding the opcode fetched from memory.

3.2 The Cycle Steps

3.2.1 Fetch

The fetch stage retrieves the instruction from memory.

- **Hardware View:** The CPU places the address in the PC onto the Address Bus. The Memory Controller reads the data at that address and places it on the Data Bus. The CPU reads the Data Bus into the Instruction Register.
- **Software View:** You access your bytecode array at the index stored in the PC.
  `uint8_t opcode = vm->memory[vm->pc];`

- **Critical Action:** The PC must be incremented immediately after the fetch so that it points to the next byte. If the current instruction requires operands (arguments), the execute stage will further increment the PC to read them.

3.2.2 Decode

The decode stage determines what the fetched value represents. In binary, 0x01 is just a number. The decoder assigns it meaning (e.g., "This is a PUSH instruction").

- **Implementation:** In C, this is ubiquitously implemented as a switch statement. The compiler transforms large switch statements into a "jump table," which is extremely efficient.
  ```
  switch (opcode) {
      case OP_PUSH: //...
      case OP_ADD:  //...
  }
  ```
  This effectively "routes" the control flow to the specific logic handler for that opcode.

3.2.3 Execute

The machine performs the actual operation.

- **Arithmetic:** The VM reads from the stack, computes, and pushes back.
- **Data Movement:** The VM reads operands from the bytecode stream (following the opcode) and moves data to the stack or memory.
- **Control Flow:** The VM modifies the PC directly, causing the *next* fetch cycle to read from a different location in the bytecode (Jumping).

3.3 Diagram: The Virtual Machine Lifecycle

```
graph TD
    Start((Start VM)) --> Init
    Init --> CheckHalt{Halt Flag Set?}
    CheckHalt -- Yes --> Stop((Terminate))
    CheckHalt -- No --> Fetch[Fetch: opcode = memory[PC]]
    Fetch --> IncPC[PC = PC + 1]
    IncPC --> Decode{Decode Opcode}
    Decode -- 0x01: PUSH --> ExecPush
    Decode -- 0x10: ADD --> ExecAdd
    Decode -- 0x20: JMP --> ExecJmp[PC = Operand Address]
    Decode -- 0xFF: HALT --> ExecHalt
    ExecPush --> CheckHalt
    ExecAdd --> CheckHalt
    ExecJmp --> CheckHalt
```

```
ExecHalt --> CheckHalt
```

4. Low-Level Memory Architecture

Designing the memory layout is the first true systems design task. Your VM is a program running inside an operating system, so its "memory" is really just memory allocated by the host OS. However, inside the VM, you must simulate the structure of a real computer's RAM.

4.1 The Harvard vs. Von Neumann Choice

There are two main ways to organize computer memory:

1. **Von Neumann Architecture:** A unified memory space where instructions (code) and data live together. This is how modern x86 PCs work. It allows code to be treated as data (self-modifying code), but it is dangerous for a simple VM because a buggy program could accidentally overwrite its own instructions with data.
2. **Harvard Architecture:** Separate memory spaces for instructions (Program Memory) and data (Data Memory). This is common in microcontrollers (like Arduino/AVR) and is much safer for a VM implementation.

Recommendation: For Lab 4, use a **Modified Harvard Architecture**.

- **Bytecode Storage:** Use a uint8_t array to store the program code. This is read-only during execution.
- **Data Storage:** Use a separate int32_t array for the stack and global variables. This separation prevents "clobbering" (corruption) of instructions and simplifies the debugging process.

4.2 The Stack Layout

The stack is a linear array, but we treat it as a dynamic LIFO structure.

- **The Array:** int32_t stack;
- **The Pointer:** int sp = 0; (or -1, depending on convention).
    - *Empty Ascending:* SP points to the next free slot. Pushing writes to stack[sp], then increments sp. Popping decrements sp, then reads.
    - *Full Descending:* SP points to the current top item. Pushing increments sp, then writes.
    - **Choice:** The "Empty Ascending" (SP points to next free) is often cleaner in C because sp also conveniently represents the "size" or "count" of items on the stack.

Bounds Checking: In systems programming, you never trust the inputs. A stack overflow (pushing beyond the array) or underflow (popping an empty stack) in your VM could crash the *host* program (the VM itself).

- **Invariant:** 0 <= sp < STACK_MAX.
- Every PUSH implementation must verify sp < STACK_MAX.
- Every POP implementation must verify sp > 0. If these checks fail, the VM should halt with a descriptive error (e.g., "Runtime Error: Stack Overflow at instruction 0x04A"). This is what distincts a robust system from a fragile toy.

4.3 Global Memory (The Data Segment)

The assignment mentions LOAD and STORE. These instructions typically refer to global variables or heap memory. Since your VM is simple, you don't need a complex malloc implementation inside the VM. You can simulate global memory with a fixed-size array.

- **Implementation:** int32_t globals;
- **Access:** STORE 0 takes the top of the stack and writes it to globals. LOAD 0 reads globals and pushes it to the stack.
- **Role:** This allows variables to persist across function calls and loop iterations, acting as the "static data segment" of a C program.

5. Implementing the Instruction Set Architecture (ISA)

The ISA is the contract between the software and the hardware. Lab 4 provides a representative set. We must translate these abstract definitions into concrete C logic.

5.1 Data Movement Instructions

These instructions move data between the "registers" (the top of the stack) and memory.

- **PUSH val (0x01):** This is an instruction with an *immediate operand*. The value to be pushed is encoded directly in the bytecode stream following the opcode.
  - *Bytecode Layout:* [0x01] (assuming 32-bit integers).
  - *Execution:* The VM reads the opcode. Then it reads the next 4 bytes, reconstructs the 32-bit integer (handling endianness!), pushes it to the stack, and increments the PC by 5 (1 for opcode + 4 for operand).
- **POP (0x02):** Discards the top element. sp--.
- **DUP (0x03):** Duplicates the top element. stack[sp] = stack[sp-1]; sp++;. This is vital for operations where you need a value for a calculation but also need to preserve it for later.

5.2 Arithmetic and Logic

These operate purely on the stack.

- **ADD (0x10):** b = pop(); a = pop(); push(a + b);
- **SUB (0x11):** b = pop(); a = pop(); push(a - b);
  - *Critical Warning:* Stack operations are non-commutative. The first value popped (b) is the right-hand operand. The second value popped (a) is the left-hand operand. If you implement push(b - a), your subtraction will be backwards (5 - 3 becomes 3 - 5).
- **CMP (0x14):** Compare a and b. Push 1 (true) or 0 (false).
  - *Implementation Detail:* b = pop(); a = pop(); push(a == b); (or < depending on spec).

5.3 Control Flow Instructions

These change the PC, allowing loops and conditionals.

- **JMP addr (0x20):** Unconditional Jump. pc = operand. The operand is an absolute address (index) in the bytecode array.

- **JZ addr (0x21):** Jump if Zero. val = pop(); if (val == 0) pc = operand;. This is how if statements work. The condition is evaluated, a boolean (0 or 1) is pushed, and JZ consumes it to decide whether to branch.

5.4 Addressing Modes

In your ISA, PUSH uses "Immediate Addressing" (the value is in the code). LOAD and STORE use "Direct Addressing" (the operand is an index into the globals array). ADD uses "Stack Addressing" (operands are implicit). Understanding these modes helps clarifying exactly where the data comes from for each instruction type.

6. Functions: The Call Stack and Stack Frames

Implementing function calls (CALL and RET) is the most challenging part of this assignment. It requires managing the flow of execution and the isolation of local variables.

6.1 The Return Address Problem

When a program executes CALL addr, it jumps to a new location. When that function finishes (RET), the VM must know where to jump *back* to. This location is the address of the instruction immediately following the CALL. Because functions can call other functions (nesting), we cannot store this return address in a single variable. We need a stack.

6.2 Approach 1: The Dual Stack Architecture

Some architectures (like Forth VMs) use two distinct stacks: a **Data Stack** for operands and a **Return Stack** for control flow.

- **CALL:** Pushes PC onto the Return Stack. Sets PC to target.
- **RET:** Pops PC from the Return Stack.
- **Advantage:** Very simple to implement. Keeps control flow separate from data.
- **Disadvantage:** Does not help with local variables. It assumes functions work only on the global data stack.

6.3 Approach 2: Unified Stack with Frames (Recommended)

Most modern systems (C, Java) use a single stack for both data and control flow. To organize this, they use **Stack Frames** (or Activation Records). This is the approach implied by "Call frames" in your requirements.

The Structure of a Stack Frame

A stack frame is a slice of the stack dedicated to one function execution. It contains:

1. **Arguments:** Values passed to the function.
2. **Return Address:** Where to go back.
3. **Saved Frame Pointer (Old FP):** The location of the caller's stack frame base.
4. **Local Variables:** Space for the function's internal data.

The Frame Pointer (FP)

While the **Stack Pointer (SP)** moves constantly as calculations happen, the **Frame Pointer (FP)** remains fixed for the duration of a function. It serves as an anchor. Local variables are accessed relative to the FP (e.g., FP + 1, FP + 2).

Implementing CALL (0x40)

When the VM encounters CALL target_addr:

1. **Push Return Address:** Push the current PC (which points to the *next* instruction) onto the stack.
2. **Push Old FP:** Push the current value of the FP register. This saves the context of the calling function.
3. **Update FP:** Set FP = SP. The current top of the stack becomes the base of the new frame.
4. **Jump:** Set PC = target_addr.

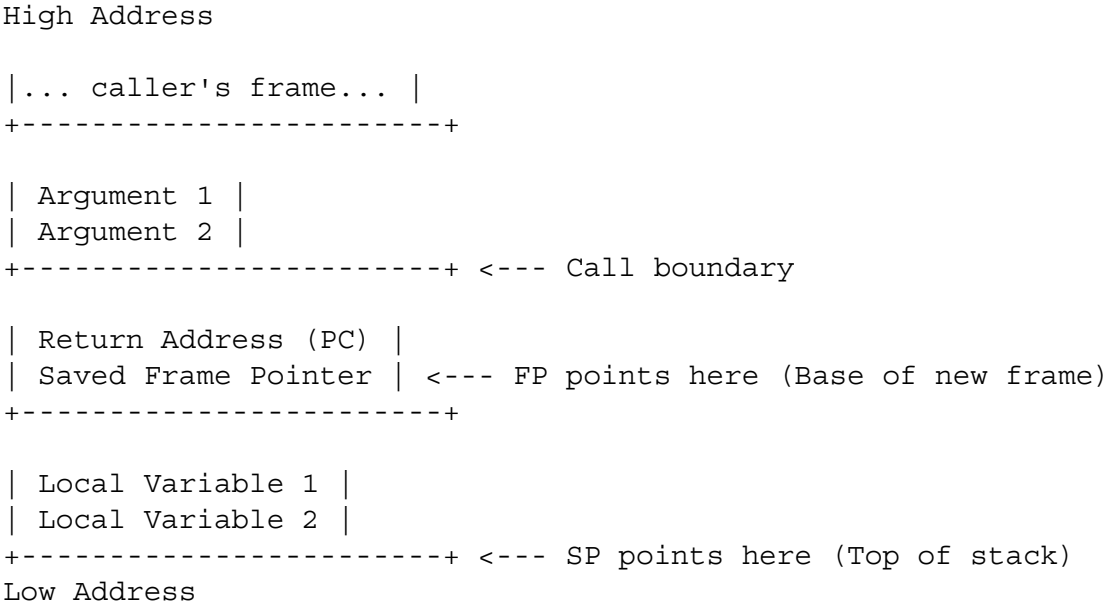
Implementing RET (0x41)

When the VM encounters RET:

1. **Clean Locals:** Set SP = FP. This instantly pops all local variables and temporary operands pushed by the current function, effectively resetting the stack height to where it was when the function started.
2. **Restore Context:** Pop the top value into FP. This restores the caller's frame pointer.
3. **Restore PC:** Pop the next value into PC. This executes the jump back.
4. **Arguments:** (Optional) If the caller pushed arguments, the RET instruction (or the caller after return) might need to pop those too.

This mechanism enables **Recursion**. Each recursive call creates a new stack frame with its own saved FP and return address. The stack grows and shrinks like an accordion.

Diagram: Stack Frame Layout



7. The Toolchain: Building an Assembler

A VM is useless without code to run. Writing raw hex bytes (0x01 0x00...) is prone to error. You need an **Assembler**: a program that translates human-readable text (Assembly) into binary bytecode.

7.1 The Two-Pass Design

The primary challenge in writing an assembler is handling **Forward References**. Consider this code:

```
    JMP end_label   ; We want to jump to 'end_label'
    PUSH 10
   ...
end_label:          ; 'end_label' is defined here
    HALT
```

When the assembler reads line 1, it encounters end_label. It has not seen the definition of end_label yet, so it does not know what address to generate for the JMP. It cannot finish the translation in a single read. This necessitates a **Two-Pass Assembler**.

Pass 1: Symbol Table Construction

The goal of the first pass is strictly to discover where every label lives.

1. Initialize a LocationCounter to 0.
2. Read the source file line by line.
3. For each instruction, identify its size in bytes (e.g., PUSH = 5 bytes, ADD = 1 byte).
4. Increment LocationCounter by that size.
5. If a label definition (e.g., start:) is found, record the pair { "start": LocationCounter } in a **Symbol Table** (a hash map or lookup array).
6. Do not generate any binary output yet.

Pass 2: Code Generation

Now that we know where every label is, we can generate the code.

1. Reset LocationCounter to 0.
2. Read the source file again.
3. Translate mnemonics to opcodes (e.g., PUSH -> 0x01).
4. If an instruction uses a label (e.g., JMP start), look up "start" in the Symbol Table.
5. Write the opcode and the resolved address to the binary output file.

7.2 The Binary File Format

You should define a formal binary structure for your .vm files. This allows the VM to verify it is running the correct type of file.

Table: Proposed Binary Format

Section	Size	Description	Value Example
Magic Number	4 Bytes	Identifier to verify file type	0xCAFEBABE
Version	2 Bytes	Assembler version	0x0001
Code Size	4 Bytes	Number of bytes in code section	0x00000100 (256)
Entry Point	4 Bytes	Initial PC value	0x00000000
Code Body	N Bytes	The raw bytecode instructions	0x01 0x05 0x10...

7.3 The Loader

Your VM needs a loader function. This function:

1. Opens the binary file.
2. Reads the first 4 bytes and compares them to the expected Magic Number. If they don't match, it aborts (preventing the VM from trying to execute a PDF or JPEG).
3. Reads the Code Size.
4. Allocates memory (malloc) for the bytecode.
5. Reads the Code Body into that memory.
6. Closes the file and returns the VM struct initialized with this data.

8. Advanced Implementation: JIT Compilation (Optional)

The "Optional Enhancement" in Lab 4 mentions JIT (Just-In-Time) compilation. While likely out of scope for a basic passing grade, implementing a rudimentary JIT demonstrates expert-level systems knowledge.

8.1 Concept

An interpreter (what we described above) is slow because it loops, fetches, and switches on every instruction. A JIT compiler eliminates this overhead by translating the bytecode into the **host machine's native code** (e.g., x86-64 machine code) at runtime and then executing it directly on the hardware.

8.2 Mechanism

1. **Allocate Executable Memory:** Standard malloc memory is not executable (due to security features like NX bit). You must use a special system call:
    - **Linux/Unix:** mmap(NULL, size, PROT_READ | PROT_WRITE | PROT_EXEC, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    - **Windows:** VirtualAlloc(..., PAGE_EXECUTE_READWRITE).
2. **Translate:** Iterate through your bytecode.
    - For ADD (VM opcode 0x10), write the x86-64 opcode for add (e.g., 0x01 0xD8 for add eax, ebx) into the mmap'd buffer.
    - For PUSH, write the x86 push instruction.
3. **Execute:** Cast the pointer to the mmap'd buffer to a function pointer void (*jit_func)() and

call it. The CPU will jump to your generated machine code and execute it natively. This transforms your VM from a software simulator into a dynamic compiler. It is difficult to debug but performs orders of magnitude faster.

9. Debugging and Visualization Strategy

Systems software is opaque. When your VM crashes, it often does so silently or with a generic Segfault. You need visibility.

9.1 The Trace Disassembler

Implement a flag -trace in your VM. When enabled, print the current state *before* every instruction execution.

```
PUSH 5 | Stack: [ 10, 20 ]
ADD | Stack: [ 10, 20, 5 ]
```

This requires a "disassembler" function that can look at memory[pc] and print the human-readable mnemonic (PUSH) instead of the hex code (0x01).

9.2 Stack Dump

Write a helper function dump_stack(VM *vm). This loops from 0 to sp and prints the contents of the stack. Call this function inside your main loop during debugging. Seeing the stack grow and shrink in real-time is the best way to catch off-by-one errors in your sp logic.

10. Study Plan and Roadmap

To successfully deliver this project, adhere to this phased schedule.

Phase 1: The Core Engine (Days 1-2)

- **Study:** Read and. Understand the switch dispatch loop.
- **Build:**
  - Define VM struct (arrays for code/stack).
  - Implement fetch, d[span_29](start_span)[span_29](end_span)ecode, execute functions.
  - Implement PUSH, HALT, and PRINT (debug instruction).
  - Hardcode a bytecode array in C uint8_t prog = { 1, 5, 255 }; and run it.

Phase 2: The Assembler (Days 3-4)

- **Study:** Read on Two-Pass Assemblers.
- **Build:**
  - Write lexer to tokenize strings.
  - Implement Pass 1 (Symbol Table) for label resolution.
  - Implement Pass 2 (Binary Generation).
  - Define your file format (Header + Magic Number).

Phase 3: Logic and Control Flow (Days 5-6)

- **Study:** Read on Program Counters.
- **Build:**
  - Implement ADD, SUB, MUL, DIV, E[span_47](start_span)[span_47](end_span)Q.

    - Implement JMP and JZ.
    - Write an assembly program that counts down from 10 to 0 using a loop.

Phase 4: Functions and Recursion (Days 7-8)

- **Study:** Read on Stack Frames and on FP/SP registers.
- **Build:**
    - Add fp register to VM.
    - Implement CALL (push PC, push FP, move FP).
    - Implement RET (restore SP, FP, PC).
    - Write a recursive factorial program to test deep stack frames.

Phase 5: Robustness and Polish (Days 9-10)

- **Build:**
    - Add bounds checking (Stack Overflow/Underflow).
    - Add the loader to read .vm files from disk.
    - Implement the -trace debugging mode.

11. Conclusion

The creation of a virtual machine is an exercise in demystification. Concepts that once seemed like magic—recursion, local variables, compilation—are revealed to be simple mechanical processes of data movement and pointer arithmetic. By choosing a stack-based architecture, you opt for a design that is elegant in its simplicity and rigorous in its adherence to computer science fundamentals.

This report has outlined the blueprint: a Harvard-architecture memory model for safety, a classic Fetch-Decode-Execute cycle for the engine, a stack-frame mechanism for function calls, and a two-pass assembler for the interface. The path is clear. Proceed methodically, validate each layer before building the next, and rely on your diagnostics (stack dumps and tracers) when the logic gets complex. You are now ready to build the machine.

Works cited

1. Stack machine - Wikipedia, https://en.wikipedia.org/wiki/Stack_machine 2. 14. Case Study: Stack Machine | Programming with MoonBit: A Modern Approach, https://moonbitlang.github.io/moonbit-textbook/stack-machine/ 3. Building a stack-based virtual machine - DEV Community, https://dev.to/jimsy/building-a-stack-based-virtual-machine-5gkd 4. language agnostic - registers vs stacks, https://stackoverflow.com/questions/164143/registers-vs-stacks 5. What are the pros and cons of register-based VMs and stack-based VMs?, https://langdev.stackexchange.com/questions/1450/what-are-the-pros-and-cons-of-register-based-vms-and-stack-based-vms 6. AskProggit: why are most VMs stack-based? : r/programming - Reddit, https://www.reddit.com/r/programming/comments/ap6wt/askproggit_why_are_most_vms_stackbased/ 7. Write your Own Virtual Machine - Justin Meiners, https://www.jmeiners.com/lc3-vm/ 8. Instruction cycle - Wikipedia, https://en.wikipedia.org/wiki/Instruction_cycle 9. Can someone provide a visualization or detailed flow of the stack frame in this Assembly MIPS code block?, https://stackoverflow.com/questions/63951945/can-someone-provide-a-visualization-or-detailed-flow-of-the-stack-frame-in-this 10. the joy of building a bytecode VM from scratch - viveknathani, https://vivekn.dev/blog/bytecode-vm-scratch/ 11. Fetch Decode Execute Cycle - IGCSE

Computer Science Revision - Save My Exams, https://www.savemyexams.com/igcse/computer-science/cie/23/revision-notes/3-hardware/computer-architecture/the-fetch-decode-execute-cycle/ 12. Let's write a bytecode interpreter in… many programming languages! | Paolo Fabio Zaino's Blog, https://paolozaino.wordpress.com/2024/07/11/lets-write-a-bytecode-interpreter-in-many-programming-languages/ 13. Separate instruction and data memory - mips - Stack Overflow, https://stackoverflow.com/questions/75367542/separate-instruction-and-data-memory 14. Unified Memory Vs. RAM: What's The Difference? | by Casenixx - Medium, https://medium.com/@casenixx_81838/unified-memory-vs-ram-whats-the-difference-9efdfddaf6e8 15. A simple stack-based virtual machine in C++ with a Forth like programming language - GitHub, https://github.com/cslarsen/stack-machine 16. How to Implement a Stack in C With Code Examples - DigitalOcean, https://www.digitalocean.com/community/tutorials/stack-in-c 17. From Scratch: How I Built a Bytecode Interpreter in C++ | by Rajat Khatri | Medium, https://medium.com/@khatrirajat888/from-scratch-how-i-built-a-bytecode-interpreter-in-c-d29be32bf350 18. Tutorial/resource for implementing VM - Stack Overflow, https://stackoverflow.com/questions/2034422/tutorial-resource-for-implementing-vm 19. CS 225 | Stack and Heap Memory - Course Websites, https://courses.grainger.illinois.edu/cs225/sp2023/resources/stack-heap/ 20. Memory Layout in C - Scaler Topics, https://www.scaler.com/topics/c/memory-layout-in-c/ 21. Chapter 6. The Java Virtual Machine Instruction Set, https://docs.oracle.com/javase/specs/jvms/se7/html/jvms-6.html 22. Fetch, decode, execute (repeat!) – Clayton Cafiero, https://www.uvm.edu/~cbcafier/cs2210/content/02_basics_of_architecture/fetch_decode_execute.html 23. How does a stack VM manage with only one stack? - Software Engineering Stack Exchange, https://softwareengineering.stackexchange.com/questions/251936/how-does-a-stack-vm-manage-with-only-one-stack 24. Explain the concept of a stack frame in a nutshell, https://stackoverflow.com/questions/10057443/explain-the-concept-of-a-stack-frame-in-a-nutshell 25. Building a stack-based virtual machine, part 6 - function calls - DEV Community, https://dev.to/jimsy/building-a-stack-based-virtual-machine-part-6---function-calls-2md5 26. tutorials on first pass and second pass of assembler - Stack Overflow, https://stackoverflow.com/questions/9210244/tutorials-on-first-pass-and-second-pass-of-assembler 27. prachidhingra09/Two-PASM: Implementation of a two pass assembler using C code to demonstrate how assembly language computation occurs. - GitHub, https://github.com/prachidhingra09/Two-PASM 28. Writing an Assembler: Part 2, https://student.cs.uwaterloo.ca/~cs241/slides/sylvie/Sylvie-L7.pdf 29. Crafting an ELF Executable | Blog - build-your-own.org, https://build-your-own.org/blog/20230219_elf_craft/ 30. JIT in C — Injecting Machine Code at Runtime | by Gamedev - Medium, https://medium.com/@gamedev0909/jit-in-c-injecting-machine-code-at-runtime-1463402e6242 31. Building a JIT Compiler from Scratch: Part 0 — How Computers Run Your Code - Medium, https://medium.com/codejitsu/building-a-jit-compiler-from-scratch-part-0-how-computers-run-your-code-3a5c71fbe88d 32. spencertipping/jit-tutorial: How to write a very simple JIT compiler - GitHub, https://github.com/spencertipping/jit-tutorial 33. Data Visualization for Better Debugging in Test Automation - BrowserStack, https://www.browserstack.com/guide/data-visualization-for-better-debugging-in-test-automation